SecureShare

Development Change Report · Prepared 5 September 2026

This report explains every change made to the SecureShare application during this development cycle. It is written for a general reader: each item says what was wrong, what was done, and why it matters, before any technical detail. Terms are explained in the glossary at the end.

At a glance

Measure	Before	After
Files modified	53 modified, 21 added, 2 removed	
Lines of code changed	3,873 added / 2,305 removed	
Automated tests	0	57 (all passing)
Code quality warnings	1,057	0
Serious security defects found	3 found, 3 fixed	

1. What the application does

SecureShare lets organisations send messages and files to each other so that **only the intended recipients can read them**. The scrambling ("encryption") happens inside the sender's own web browser before anything is uploaded. The company running SecureShare stores only scrambled data and holds no key that could unlock it.

This design is called *zero-knowledge*. Its value depends entirely on the promise being true in practice, which is why most of the work below concerns keeping that promise airtight.

2. Major changes

2.1 Private keys were being sent to users' browsers (CRITICAL)

In plain terms: every user was being handed the locked spare keys of everyone they had ever messaged.

Each user has a personal key that unlocks their messages. A copy is stored on the server in locked form, protected by the user's master password. Because of the way the database was being queried, that locked key — along with the values needed to attack it — was being included in the data sent to other users' browsers.

Anyone could have taken a colleague's locked key from their own page, then tried passwords against it privately on their own computer, with no limit on attempts. A weak password would have given them that person's key, and with it every message ever sent to them.

Fixed by listing explicitly which fields may leave the server, in two places: the dashboard and the profile page. An automated test now fails if key material is ever added back.

2.2 Attachment file names were readable to the company

In plain terms: a file called "Redundancy List Q3.xlsx" gave away the secret even though the file itself was locked.

Message text was scrambled, but file names were stored as ordinary text and shown in the interface before anything was unlocked. A file name is often as revealing as the file. The name was also given to the storage provider.

Fixed by scrambling the file name with the same key as the file, and giving the uploaded file a meaningless random name. The interface now shows a padlock and "Encrypted name" until the recipient unlocks the message. Older files still display correctly.

2.3 "Expiring" messages did not actually expire

In plain terms: setting a message to expire hid it, but the file stayed on the internet forever.

The expiry setting was applied to the message but never to its attachments, and nothing ever deleted the stored file. A message could pass its expiry date while its attachment remained downloadable indefinitely.

Fixed by applying expiry to attachments, checking it when a file is requested, and adding a scheduled clean-up task that permanently deletes the stored file. Files are also deleted once every recipient has deleted the message. Expiry now means the data is gone, not hidden.

2.4 Administrator access was invisible

In plain terms: an administrator could read any company message and leave no trace.

The organisation vault lets administrators open messages when the original recipient is unavailable — a legitimate feature. But no record was kept when they used it. The application also advertised a "sign-in" entry in its activity log that was never actually written.

Fixed by recording every vault access with the administrator's identity, the message, and the time; and by genuinely recording sign-ins with their location and browser. The vault is now a supervised tool rather than a silent back door.

2.5 Password protection was below current standards

In plain terms: guessing a stolen password was about six times cheaper than it should have been.

The master password protects the user's key. Two weaknesses: users could choose an eight-character password with no strength requirement, and the protective calculation applied to it used 100,000 rounds — a figure from 2013. Current guidance is 600,000.

Fixed by requiring at least twelve characters with mixed character types, adding a live strength meter, and moving to 600,000 rounds. Because existing accounts were created under the old setting, the number used is now recorded per account, so current users keep working while new keys get the stronger protection.

2.6 Anyone could query the user directory

In plain terms: a stranger could ask the site which email addresses had accounts, and list every organisation.

Several internal operations were reachable without being signed in. They could be used to confirm whether a given email address was registered and to enumerate every organisation on the platform — useful groundwork for a targeted phishing campaign.

Fixed by requiring a valid session on every one of them, and adding limits on how often a

signed-in user may call the expensive ones. This also protects the sending reputation of the company's email domain from abuse.

2.7 The activity log gave confidently wrong answers

In plain terms: searching the log checked the most recent fifty entries and reported the result as though it had checked everything.

Searching or filtering the audit log only examined the entries already loaded on screen. During an investigation this would report "2 events" when the true answer might be forty-seven. A tool used to answer "who did what" must not do this.

Fixed by performing searches and filters in the database across the whole log, and reporting a true total. The page also gained date filtering, grouping by day, and a spreadsheet export.

2.8 Defences added around the browser

Because SecureShare unlocks messages inside the browser, the page itself must be protected from tampering. A Content Security Policy now restricts what may run on each page, along with protections against the site being embedded in a fraudulent page.

This was tested rather than assumed. The first version would have made the application fail to load; the policy was adjusted and verified page by page before being kept.

2.9 Automated testing and continuous checking

The project previously had no automated tests. There are now 57, covering the encryption functions, password rules, rate limits, and the safeguards protecting the data-export and private-key paths. A continuous integration pipeline runs these checks, plus type checking, code-quality checks and a full build, on every change.

2.10 Plain-language rewrite of all text

The wording throughout described the cryptography rather than the situation. One error read *"Cryptographic Mismatch Detected — The assigned organization does not match the recipient's secure record."* It now reads *"ana@bolt.legal is at Bolt Legal, not Acme Corp."*

Every screen, error message, notification and marketing page was rewritten. A branding

inconsistency was also corrected: sign-in code emails were branded "SecureMail" while the site said "SecureShare" — the exact mismatch people are trained to treat as a phishing signal.

2.11 Visual redesign

The interface was tidied without changing its layout, its behaviour on phones, or its colours. Decorative elements were removed in favour of consistency: 47 all-capital micro-labels, 27 uses of the heaviest available text weight, 49 pieces of text below readable size, five competing corner shapes, and 40 padlock and shield icons used as decoration. Icons now appear only where they carry meaning.

2.12 Dashboard rebuilt in a familiar layout

The mailbox was restructured to follow the conventions of a mainstream email client: a full-width bar with search, a labelled sidebar with a Compose button and unread count, and messages that open full-width. Because subjects stay locked until opened, message rows show what is genuinely known — sender, attachments, recipients — rather than repeating the word "Encrypted".

2.13 Profile and organisation pages restructured

The profile moved to a settings-style layout with sections down the left. Security settings were regrouped by task — Password, Keys, Devices. Members and pending invitations, previously spread across four separate panels, became a single list with invitations sent from a dialog.

2.14 Landing page expanded

Three sections were added: an honest account of what the company can and cannot see, a comparison against ordinary email attachments, and a set of frequently asked questions. Every claim was checked against how the software actually behaves. Signed-in visitors now see a link to their dashboard instead of an invitation to sign up.

3. Minor changes

Area	Change

Database connections	Development server opened a new database connection on every code reload until it ran out. Now reuses one.
Database speed	Twelve indexes added to the queries used on every page load.
Message search	Searching by subject could never match, because subjects are scrambled. Now searches the subjects unlocked in the session, and shows them in the list.
Email addresses	Sending to an address typed with capital letters failed with "recipient not found". Now matched consistently.
Expired messages	Were still listed in the mailbox, and only revealed as expired when opened. Now hidden.
Organisation vault	The vault view could not load additional pages or receive new messages.
Copied recipients	A copied recipient could be left without an organisation, quietly excluding their administrators from recovery. Now required.
Compose window	Adding a recipient while an address lookup was in progress could delete the row. Also, a missing subject blocked sending with no message shown.
Recovery key	Accounts without a recovery key reported "invalid recovery key" instead of explaining that none was set up.
Deleting a message	The quick-delete button on a message row deleted immediately. Now asks for confirmation.
Keyboard and screen readers	All nine dialogs gained proper labelling, keyboard trapping and Escape-to-close. Icon-only buttons gained descriptions.
Spreadsheet export safety	Exported logs contain user-supplied names. Values that spreadsheets would treat as formulas are now neutralised.

Audit log detail	Entries recorded which message or file was involved but never displayed it. Now shown. A field storing an unreadable scrambled file name was removed.
Setup documentation	The project guide was the unmodified framework template. Replaced with real instructions, and a template for the required settings was added.
User guide corrections	The guide instructed users to choose an eight-character password the software now rejects; claimed sign-ins were recorded when they were not; and described a sending limit that did not exist.
Code quality tooling	The quality checker was reporting 1,057 problems, almost all from auto-generated files it should have ignored. Now reports zero.
Unused code	An entire unreachable upload page and its 230-line form were removed.
Application icon	The generic shield was replaced with a purpose-drawn envelope-and-keyhole mark used consistently throughout.
Mobile compose	Moving Compose into the sidebar removed it from phones. A floating button was added.

4. How the work was verified

- **Automated tests** — 57 checks covering encryption, password rules, rate limiting and export safety.
- **Type checking** — the whole project verified for internal consistency. This caught two crashes hidden by loose typing.
- **Code quality checks** — zero warnings across the project.
- **Full build** — the production build completes without error.
- **Live inspection** — pages were loaded from a running server and read as a visitor sees them. This is how an outdated marketing panel and a security policy that would have broken the site were both caught; neither was visible in the code alone.
- **Database verification** — schema changes were applied and confirmed against the live database.

5. Outstanding items

The following were identified but not carried out, and are recorded here for completeness.

Item	Note
Key stored on device	The unlocked key is held in browser storage so messages open without re-entering the password each time. Convenient, but it means physical access to an unlocked computer is enough. A deliberate trade-off worth revisiting.
Forward and reply	Carry the subject only, not the original message or its attachments.
Attachment preview	Images and documents must be downloaded; they cannot be viewed in place.
Repeated unlocking	Moving away from a message and back requires unlocking it again.
Sending to new people	Recipients must already have an account. They now receive a clear explanation rather than an error.
Sign-in notifications	Sent on every sign-in, which some users may find excessive.
Interface testing	Automated tests cover the underlying logic. The signed-in screens have not been through an automated click-through and are worth reviewing by hand.

6. Glossary

Term	Meaning
Encryption	Scrambling information so only someone with the right key can read it.
Key	The secret value that unscrambles information. Each user has their own.

Master password	The password protecting a user's key. Never sent to the company, and therefore impossible for them to reset.
Recovery key	A one-off code shown at sign-up. The only way back in if the master password is forgotten.
Zero-knowledge	A design where the company operating the service cannot read what users store on it.
Audit log	A permanent record of who did what, and when.
Organisation vault	A shared key letting administrators open company messages when the recipient is unavailable. Every use is recorded.
Content Security Policy	A browser instruction limiting what code a page may run, reducing the damage a successful attack can do.
Continuous integration	Automatic checking of every change before it reaches the live site.
Metadata	Information about a message rather than its contents — who sent it, when, and how large. Encryption hides contents, not this.

Report generated 5 September 2026. Figures taken from the project's own test, build and quality tooling at the time of writing.